.Scora

Trust Skills, Not Portfolios.

System Blueprint & Technical Planning

MVP architecture, modules, flows, data model, scoring, and security

Deliverable 04

Product: Scora **Team: Code Luck**
Phase: Product Validation & Strategy **Date: 27 July 2026**

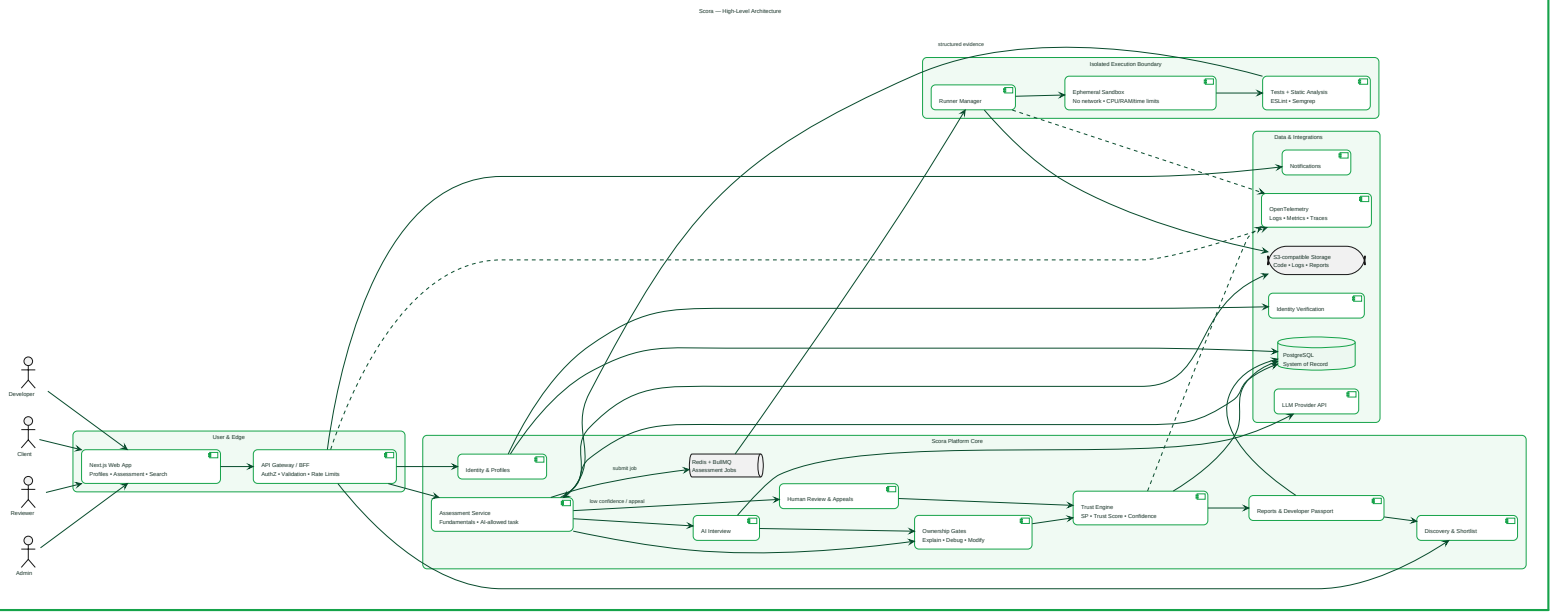
1. Technical goals and principles

Principle	Design consequence
Ownership before verification	Working output is insufficient; required explanation, debugging, and modification gates must pass.
Explainable and versioned	Store scoring-model version, component values, confidence, recency, and reason codes.
AI-aware, not AI-blind	Use both AI-restricted fundamentals and AI-allowed engineering tasks; tool use alone is not a failure.
Secure by isolation	Submitted code runs outside the main application with strict resource, network, and filesystem limits.
Start modular, not microservice-heavy	Use a modular monolith for product speed; isolate code execution and asynchronous workers as separate trust boundaries.
Privacy by design	Purpose-built challenges, minimal retention, explicit consent, access controls, and audit trails.

2. High-Level Architecture Diagram

High-Level Architecture

PlantUML - vector quality

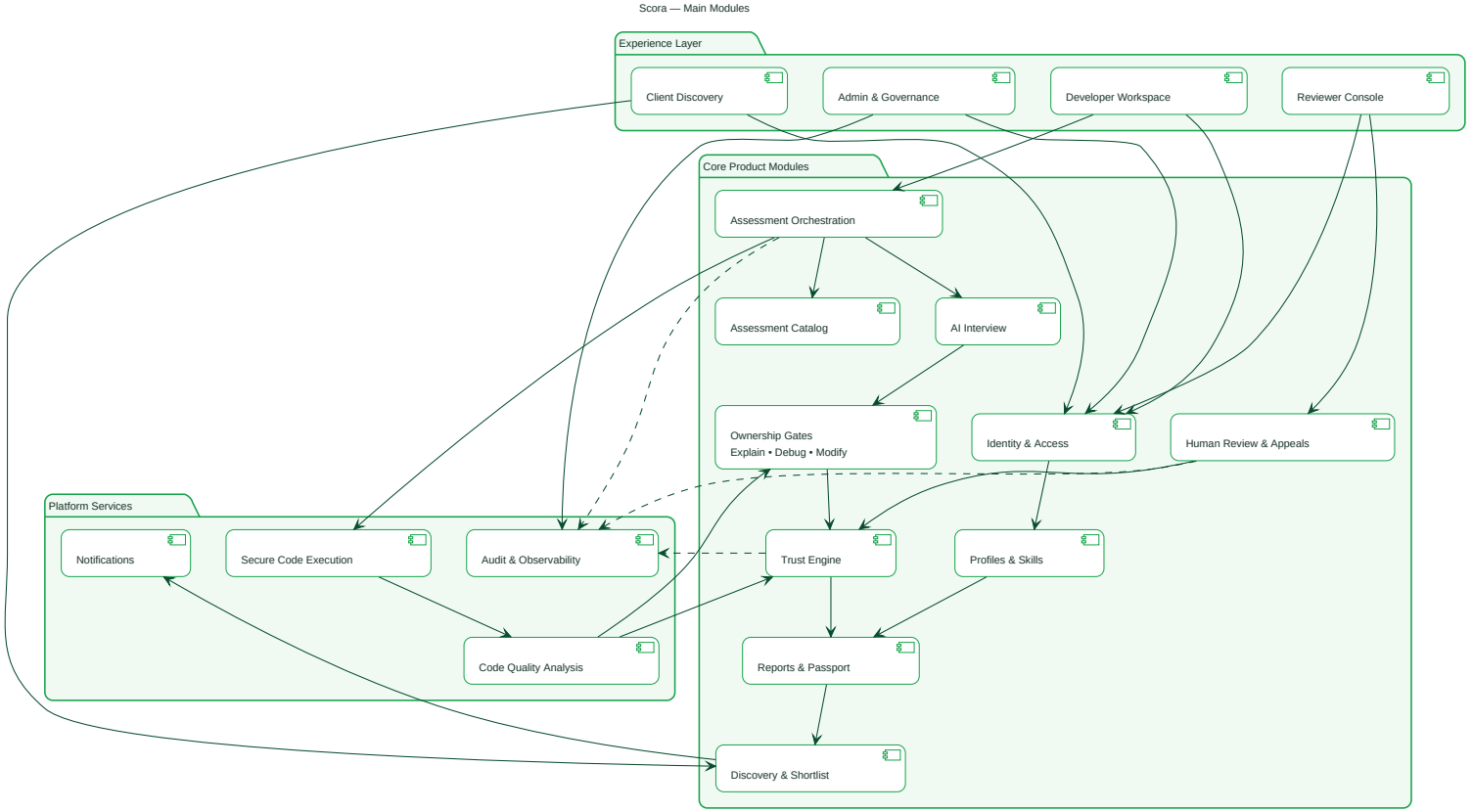


Architecture decision: the MVP uses a Next.js experience layer, a NestJS modular backend, PostgreSQL, object storage, Redis/BullMQ, and isolated execution workers. Verification is a state machine with mandatory ownership gates; a high automated code score cannot bypass failed understanding evidence. The source diagram is included as PlantUML in the Technical folder.

3. Main modules

Main Modules

PlantUML - vector quality



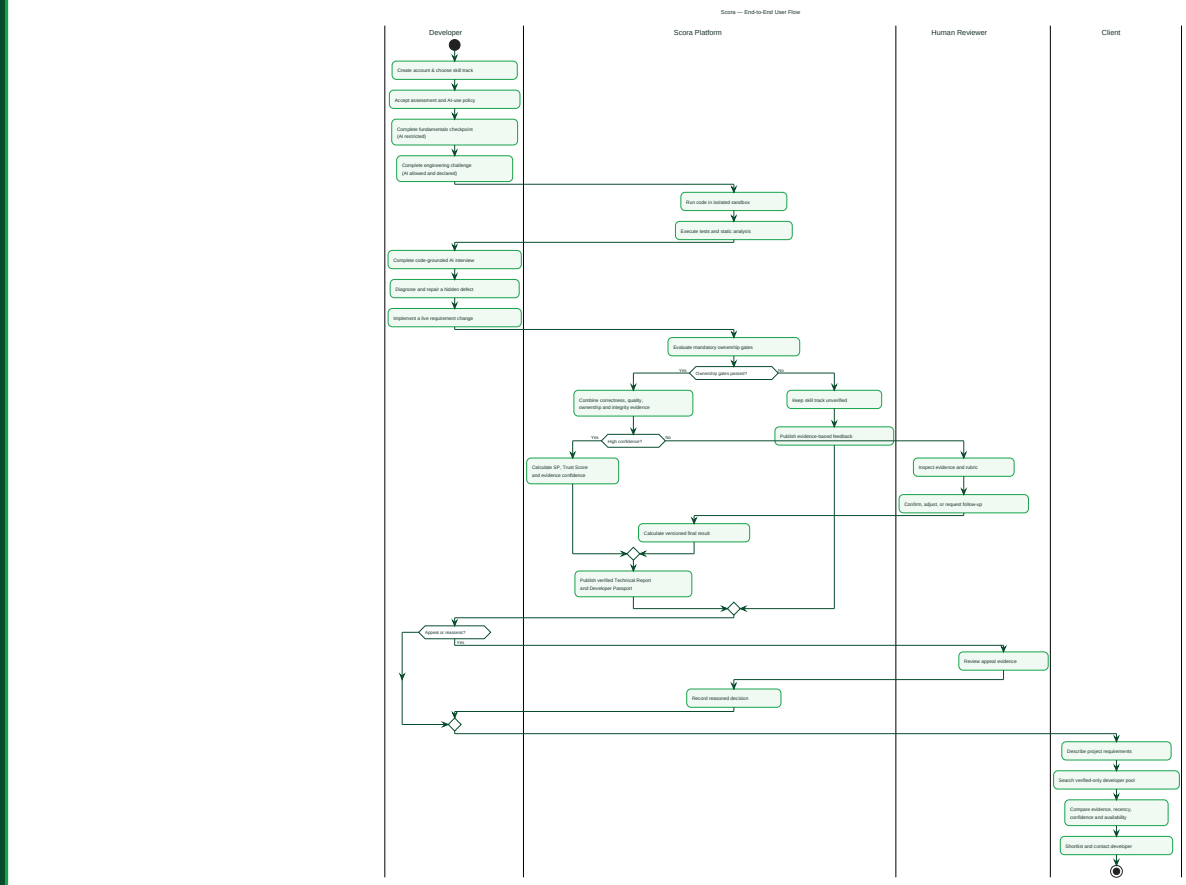
Module group	Included modules	Role in the MVP
Experience	Developer Workspace; Client Discovery; Reviewer Console; Admin & Governance.	Each role sees a focused workflow rather than one overloaded dashboard.
Assessment	Fundamentals; AI-allowed task; Secure Execution; Quality Analysis; Code-grounded Interview; Live Change.	Measures both delivery ability and whether the candidate owns the reasoning behind the code.

Module group	Included modules	Role in the MVP
Trust	Understanding Gates; Human Review & Appeals; Trust Engine; Reports & Passport.	Blocks track verification when required ownership evidence fails, then scores the quality of passing evidence.
Marketplace	Profiles & Skills; Discovery & Shortlist; Notifications.	Connects verified evidence to a real client decision without adding escrow to the MVP.
Platform	Identity & Access; Audit & Observability.	Protects roles, consent, traceability, operational investigation, and governance.

4. User flow overview

User Flow

PlantUML activity diagram

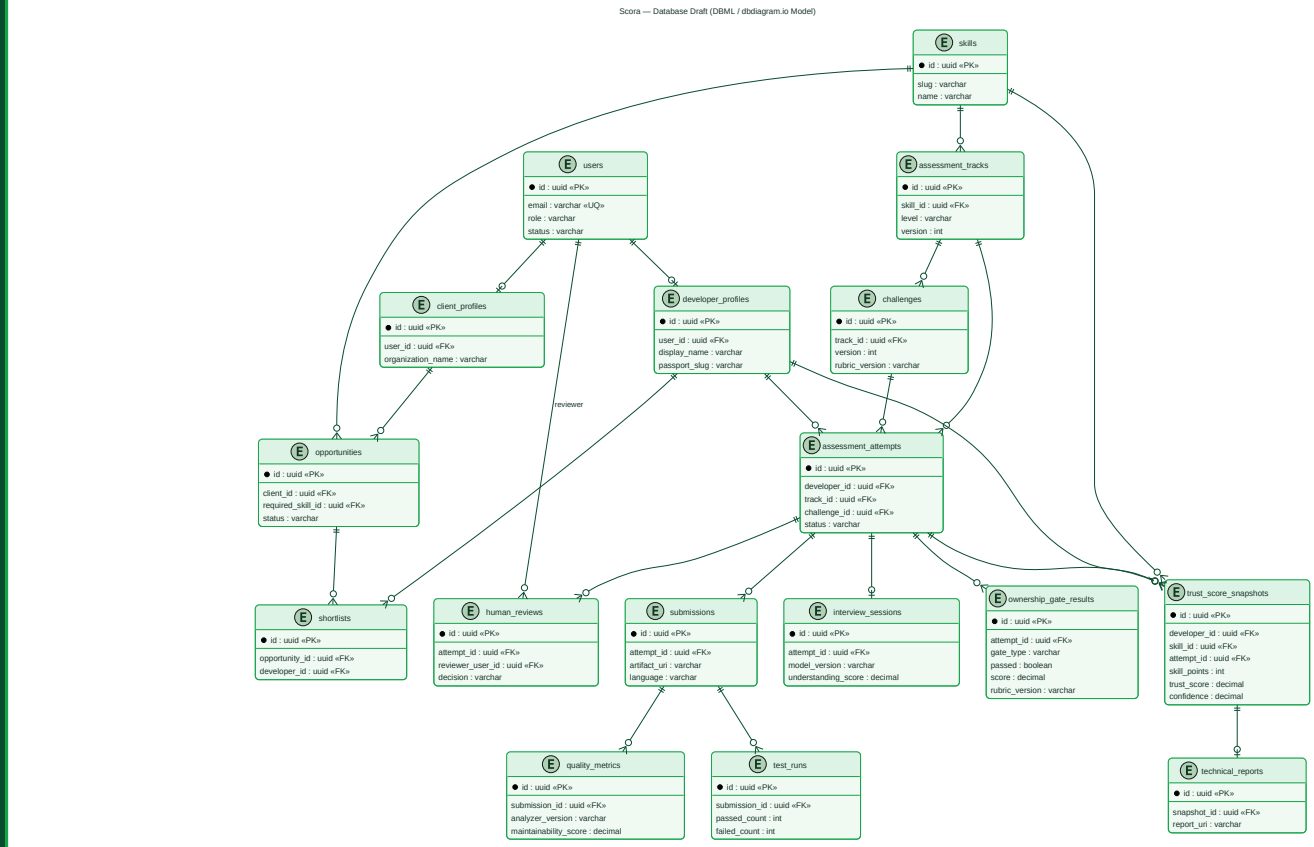


Developer journey: pass a fundamentals checkpoint, complete an AI-allowed engineering task, explain the submitted code, diagnose a hidden defect, and implement a live requirement change. Failure of a required understanding gate leaves the track unverified even if the original code runs. Client journey: search only verified profiles, compare evidence and confidence, then shortlist and contact.

5. Database draft

Database Draft

DBML / dbdiagram.io model



Open the interactive Scora ERD on dbdiagram.io (the complete DBML source is included in the Technical folder).

5.1 Database design notes

Domain	Core tables	Design notes
Identity	users, roles, consents, verification_records	Role, status, identity level, consent version; keep PII separated.
Profiles	developer_profiles, client_profiles, skills, developer_skills	Public fields separated from private assessment data.
Assessment catalog	tracks, assessment_templates, challenges, challenge_versions, rubrics	Immutable versions once attempts begin.
Attempts	assessment_attempts, submissions, test_runs, activity_events	Attempt state machine, timestamps, artifacts, integrity signals.
Analysis	quality_metrics, interview_sessions, ownership_gate_results, evaluator_outputs	Store each explanation, debugging, and live-change gate result with evidence, confidence, and rubric version.
Review	human_reviews, review_assignments, appeals	Reviewer identity, decision, evidence, reason codes, conflict disclosure.
Scoring	score_components, trust_score_snapshots, skill_point_events	Never store only a final number; include model version and confidence.
Reports	technical_reports, passports, report_access_logs	Audience-specific views and controlled sharing.
Marketplace	opportunities, shortlists, contacts	Keep contract/payment scope optional for MVP.
Governance	audit_logs, policy_versions, incidents	Append-oriented audit records and security response.

6. Trust Engine v1

Scoring model

Scora first applies mandatory verification gates: code-specific explanation, hidden-bug diagnosis, and live modification. A failed required gate means the skill track remains unverified. Weights below rank the strength of evidence only after the minimum ownership standard is met.

Component	Initial weight	Example evidence
Correctness & task completion	25%	Deterministic tests, hidden tests, failure modes.
Code quality & engineering	25%	Structure, readability, maintainability, testing, security findings.
Technical ownership	35%	Code-specific explanation, trade-offs, hidden-bug repair, and live requirement change.
Human verification	10%	Rubric-based review for sampled, appealed, high-stakes, or uncertain attempts.
Process integrity	5%	Identity and anomaly review; flags require review rather than automatic guilt.

Verification status answers whether the candidate met the minimum ownership standard. Skill Points (SP) represent demonstrated capability among verified evidence. Trust Score represents reliability, confidence, recency, and review strength; it is not a moral judgment or a universal ranking.

- Confidence: increases with sufficient, consistent, independently supported evidence.
- Recency: older evidence gradually contributes less, with a visible assessment date.
- Consistency: repeated, comparable evidence reduces uncertainty but must not permanently punish learning.
- Versioning: every snapshot stores the algorithm, rubric, challenge, evaluator, and policy version.
- Appeals: users can challenge factual or procedural errors; overrides require reason codes and audit history.
- Integrity signals: used to trigger review, not to produce an irreversible automatic accusation.

7. Secure code-execution boundary

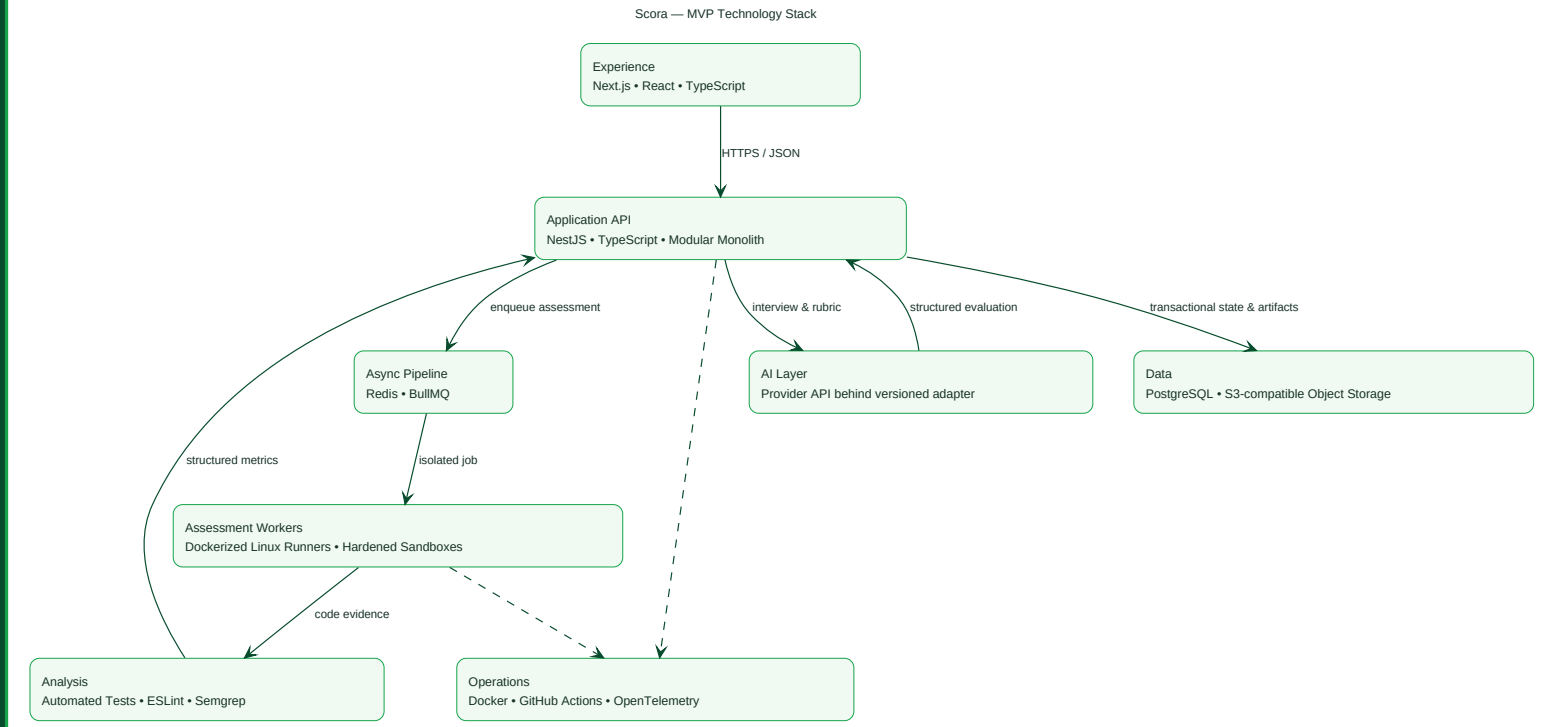
Control	MVP requirement
Isolation	Ephemeral container or microVM per run; non-root user; no privileged mode; read-only base filesystem.
Resources	Strict CPU, memory, process, file-size, and wall-clock limits; kill on timeout.
Network	No outbound network by default; allowlist only when a challenge explicitly requires it.
Filesystem	Temporary workspace only; no host mounts, secrets, sockets, or shared writable volumes.
Images	Signed, minimal, patched runtime images; fixed versions per challenge.
Input/output	Validate filenames and parameters; cap logs; treat artifacts as untrusted.
Queue	Separate worker identity and credentials; idempotent jobs; retry and dead-letter handling.
Monitoring	Execution audit, anomaly alerts, sandbox-escape incident procedure, regular security testing.

Docker's security guidance emphasizes namespaces, cgroups, capabilities, and daemon protection, while warning that configuration and daemon access are critical attack surfaces. Before public production, we benchmark hardened containers against microVM and managed sandbox options.

8. Tech Stack Justification

Technology Stack

PlantUML - vector quality



Layer	What Scora uses	Why it fits the MVP
Web	Next.js + TypeScript	Fast product iteration, typed UI, server/client rendering options, mature ecosystem.
Backend	NestJS + TypeScript, modular monolith	Clear module boundaries, validation, background-job integration, shared language with frontend.
Database	PostgreSQL	Strong relational model, transactions, indexing, reporting, and JSON support where needed.
Queue / cache	Redis + BullMQ	Async assessment pipeline, retries, status updates, and rate limiting.

Layer	What Scora uses	Why it fits the MVP
Artifacts	S3-compatible object storage	Submission bundles, logs, reports, retention policies, and signed access.
AI	Provider API behind an abstraction layer	Validate the product before custom-model investment; keep prompts, models, and evaluations versioned.
Analysis	Language-specific linters + Semgrep; later SonarQube	Deterministic and explainable signals complement AI review.
Execution	Dedicated Linux workers; hardened containers initially	Separate trust boundary and reproducible language images.
Observability	OpenTelemetry + managed logs/metrics	Trace long async workflows and investigate assessment incidents.
Delivery	Docker, CI/CD, infrastructure-as-code later	Reproducible environments and controlled releases.

9. MVP delivery slices

Slice	Outcome	Exit criteria
0 - Research prototype	Clickable report and assessment concept.	Clients understand evidence; developers accept time/effort.
1 - Assessment core	One track, deterministic tests, submissions, basic report.	30-50 complete attempts; stable execution and useful feedback.
2 - Understanding	Code-grounded AI interview and structured rubric.	Expert agreement and repeatability reach a defined threshold.
3 - Trust Engine	Versioned SP, Trust Score, confidence, and recency.	Calibration set reviewed; score explanations pass comprehension tests.
4 - Client discovery	Search, filters, reports, shortlist, contact.	Pilot clients can produce qualified shortlists faster.
5 - Governance	Appeals, reviewer calibration, fairness and security dashboards.	Documented operating process before broader launch.

10. Technical risks and open decisions

Decision / risk	Scora decision
Container vs microVM	Prototype with hardened dedicated workers; benchmark managed sandbox/microVM isolation before public scale.
AI evaluator reliability	Require structured rubrics, deterministic evidence, confidence thresholds, and sampled human calibration.
Challenge leakage	Version pools, parameterization, rotation, monitoring, and post-leak invalidation.
Cross-language fairness	Launch one or two tracks; never compare uncalibrated tracks as equivalent.
Behavioral monitoring	Defer invasive surveillance; prioritize transparent integrity signals and user consent.
Proprietary code	Do not request production repositories in the MVP; use purpose-built challenges.
Score misuse	Expose intended use, confidence, limits, expiration, and a non-discrimination policy.

References

We selected sources for their direct relevance and authority. Company pages describe current product behavior; surveys and research papers support market and risk claims. Assumptions that still require interviews or field testing are identified in the validation sections.

1. Stack Overflow 2025 Developer Survey - AI. Among respondents answering the accuracy question, 46% distrust AI-tool accuracy versus 33% who trust it. The most reported frustration is output that is almost right, while 45% cite extra time debugging AI-generated code.
https://survey.stackoverflow.co/2025/ai

2. GitHub randomized controlled study on Copilot and code quality. A study with 202 developers who had at least five years of experience found improved functionality and modest quality gains with Copilot. AI use alone is therefore not evidence of low skill.
https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/

3. Do Users Write More Insecure Code with AI Assistants?. In a security-focused user study, participants with an AI assistant produced less secure code and were more likely to believe their code was secure; lower trust and more active engagement correlated with fewer vulnerabilities.
https://arxiv.org/abs/2211.03622

4. Lost at C - USENIX Security 2023. A different user study found only a small security impact in its specific C task. This mixed evidence shows that risk depends on the task, user, and verification process rather than AI use alone.
https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval

5. METR - Early-2025 AI and experienced developer productivity. A randomized study of experienced open-source developers working in familiar repositories found early-2025 AI tools slowed completion in that setting, despite participants expecting a speedup.
https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

6. Generative AI code tools and software-engineer hiring. A study of 32 industry professionals reports that GenAI complicates evaluation of candidates' true abilities, while many participants value candidates who can demonstrate effective and informed AI use.
https://arxiv.org/abs/2409.00875

7. OWASP Secure Coding with AI. OWASP recommends independent verification, full-diff review, adversarial testing, dependency checks, human accountability, and sandboxed execution for AI-assisted development.
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html

8. Upwork Job Success Score. Upwork's Job Success Score reflects client satisfaction, contract outcomes, and long-term client relationships. It is a reputation signal rather than a direct test of code ownership or current engineering understanding.
https://support.upwork.com/hc/en-us/articles/38437362753171-What-is-a-Job-Success-Score

9. Upwork - Build a complete profile. Upwork emphasizes work history, certifications, availability, achievements, keywords, and portfolios as profile trust inputs.
https://support.upwork.com/hc/en-us/articles/34924882793107-Build-a-100-complete-profile

10. HackerRank Screen - AI-era skills and integrity. HackerRank separates fundamentals without AI, ability to work with AI, and AI concepts, while adding identity and assessment-integrity controls. This validates multi-mode assessment in the AI era.
https://www.hackerrank.com/products/screen

11. Codility Task Library. Codility combines real-life skills, problem solving, and technical knowledge tasks, supporting a rounded assessment rather than a single puzzle score.
https://support.codility.com/hc/en-us/articles/360043827713-The-Codility-Task-Library

12. Docker Engine security. Docker documents namespaces, cgroups, capabilities, daemon attack surface, and hardening controls relevant to isolated code execution.
https://docs.docker.com/engine/security/